

PROMPT LOG & MINH CHỨNG PHÁT TRIỂN

FocusGuard - AI Tài Năng Việt 2026 - Bảng A THCS

Đội iKH Future Makers | Mã đội AI26A0049 | Nguyễn Hoàng Bảo Long - Lớp 8/1, THCS Tây Nha Trang

Tính trung thực của minh chứng: Các prompt dưới đây được lấy trực tiếp từ tài liệu “một số câu lệnh” trong thư mục minh chứng do đội cung cấp. Nội dung prompt được giữ nguyên, không dựng lại để làm đẹp.

1. Các prompt thực tế đã sử dụng

Prompt 1

“migrate lại data đi sao lúc tui chạy cmd để chạy web lại reset data trong admin v lúc đó tui xóa tài khoản ảo r tạo tài khoản mới xong tắt cmd tui mở lại r reset như ban đầu v sao ko lưu data”

Prompt 2

“# 🚫 KHÓA PHẠM VI – CHỈ TỐI ƯU CAMERA

Hãy tiếp tục triển khai **kế hoạch nâng cấp hiệu năng Camera của FocusGuard AI**.

⚠️ ****CỰC KỲ QUAN TRỌNG:**** Task này ****CHỈ**** tập trung vào camera, FPS, tốc độ xử lý AI và video stream**.

Không được tự ý mở rộng phạm vi sang các hệ thống khác.

❌ **TUYỆT ĐỐI KHÔNG SỬA**

Trong task này **KHÔNG** thay đổi logic của:

* `student\_id` / `track\_id`

* Logic nhận diện danh tính khuôn mặt

* Công thức tính Focus Score

* Mock / Fake / Simulated Students

* Database / schema database

* Login / Register / Authentication

* Class Membership

* Attendance

* Analytics / Reports

* Seating / Seat Assignment

* UI redesign / giao diện mới

Nếu phát hiện bug ở các phần trên thì ****chỉ ghi nhận trong báo cáo cuối****, không sửa trong task này.

Các phần này sẽ được xử lý ở những task riêng sau.

✅ **PHẠM VI ĐƯỢC PHÉP SỬA**

1. Tối ưu Camera Capture

Hiện tại cần kiểm tra xem camera capture có đang bị AI inference làm chậm hay không.

Mục tiêu:

```text

Camera

↓

Camera Capture Thread

↓

Latest Frame Buffer

↓

AI Processing

↓

AI Result Cache

↓

Stream Renderer

↓

Browser

```

Yêu cầu:

- * Camera đọc frame liên tục bằng thread riêng.
- * Chỉ giữ frame mới nhất.
- * Buffer tối đa khoảng 1 frame.
- * Không xếp hàng hàng trăm frame cũ.
- * Nếu AI chưa xử lý xong frame trước thì bỏ frame cũ và lấy frame mới.
- * Không để camera phải chờ YOLO / MediaPipe / InsightFace.

Mục tiêu thực tế:

****Camera/Stream khoảng 25–30 FPS nếu phần cứng cho phép.****

Không được chỉ đặt:

```
```python
CAP_PROP_FPS = 30
```
```

rồi kết luận camera chạy 30 FPS.

Phải đo FPS thực tế.

2. Tách AI Processing khỏi Video Stream

Đây là phần quan trọng nhất.

Hiện tại nếu:

```
```text
get_frame()
→ đọc camera
→ YOLO
→ ByteTrack
→ Phone detection
→ InsightFace
→ MediaPipe
→ tính toán
→ vẽ
→ JPEG
→ trả frame
```
```

thì stream sẽ bị chậm theo AI.

Hãy chuyển sang:

```
```text
Camera Thread
↓
Latest Frame
↓
AI Worker
↓
Latest AI Results
↓
Stream Renderer
↓
Browser
```
```

Browser luôn lấy frame mới nhất.

AI xử lý chậm hơn camera cũng không được làm video bị đứng.

3. Tối ưu YOLO

Audit toàn bộ nơi YOLO được gọi.

Đặc biệt kiểm tra trường hợp YOLO đang chạy nhiều lần trên cùng một frame:

```
```text
Person Detection / ByteTrack
+
Phone Detection
```
```

Nếu có thể dùng chung inference một cách an toàn thì tối ưu.

Nhưng:

⚠ Không được gộp pipeline một cách mù quáng nếu làm hỏng ByteTrack hoặc hành vi hiện tại.

Nếu không thể gộp an toàn:

- * Giữ Person + ByteTrack.

- * Giảm tần suất Phone Detection.

- * Cache kết quả phone detection giữa các frame.

Không được xóa tính năng phát hiện điện thoại.

4. Tối ưu Face Recognition

⚠ KHÔNG thay đổi logic nhận diện danh tính.

Chỉ tối ưu tốc độ thực thi.

Nếu hiện tại InsightFace đang xử lý:

```
```text
```

Full Camera Frame

```
```
```

hãy kiểm tra khả năng sử dụng:

```
```text
```

Person / Face Region

↓

Crop khuôn mặt

↓

InsightFace

```
```
```

để giảm lượng dữ liệu phải xử lý.

Có thể giảm tần suất recognition hoặc cache kết quả nếu an toàn.

Nhưng phải giữ nguyên:

- * student matching hiện tại

- * threshold hiện tại

- * identity behavior hiện tại

- * API hiện tại

****Không thay đổi kiến trúc `student\_id` / `track\_id` trong task này.****

5. Tối ưu MediaPipe

Giữ nguyên toàn bộ chức năng MediaPipe hiện tại.

Không chạy MediaPipe không cần thiết trên mọi frame.

Có thể sử dụng:

- * throttling

- * scheduler

- * round-robin

- * cache kết quả

Ví dụ:

```
```text
```

Camera: 30 FPS

MediaPipe:

không nhất thiết 30 FPS

→ xử lý một số frame

→ giữ kết quả

→ cập nhật kết quả mới

```
```
```

Không thay đổi ý nghĩa của:

- * Head Pose

- * EAR

- * Drowsiness

- * Face landmarks

Chỉ tối ưu cách chạy.

6. Tối ưu JPEG Encoding

Kiểm tra thời gian:

```
```text
```

AI Processing

+  
OpenCV Drawing  
+  
JPEG Encoding  
...

Không để JPEG encoding làm nghẽn toàn bộ camera pipeline.

Nếu phù hợp:

- \* cache kết quả
- \* encode frame ở stream thread
- \* giảm thao tác dư thừa
- \* tránh encode lại những thứ không cần thiết

Nhưng vẫn phải giữ stream hiện tại hoạt động bình thường.

---

## # 7. Thread Safety

Khi tách thread phải đảm bảo an toàn cho dữ liệu dùng chung.

Đặc biệt:

```text  
latest_frame
latest_ai_result
camera state
AI result cache
```

Không được xảy ra:

- \* race condition
- \* deadlock
- \* frame bị hỏng
- \* memory leak
- \* queue tăng vô hạn
- \* camera không release được

---

## # 8. Tối ưu Socket.IO

Kiểm tra các event realtime.

Không gửi dữ liệu quá dày nếu không cần thiết.

Ưu tiên:

```text  
Camera frame → stream riêng
Metadata / AI state
→ gửi theo interval hợp lý hoặc khi state thay đổi
```

Không gửi toàn bộ dữ liệu nặng ở mọi frame.

⚠ Không thay đổi cấu trúc nghiệp vụ của Socket.IO nếu không cần thiết.

---

## # 9. Adaptive AI Processing

Có thể áp dụng mức xử lý phù hợp với số người trong camera.

Ví dụ:

### ### 1–3 người

```text  
YOLO: 12–15 FPS
MediaPipe: 8–10 Hz/người
```

### ### 4–10 người

```text  
YOLO: 10–12 FPS
MediaPipe: xử lý luân phiên
```

### ### 11–30 người

```text  
YOLO: 8–10 FPS
MediaPipe: xử lý luân phiên
```

---

Camera/stream vẫn cố gắng duy trì khoảng:

```text

20–30 FPS

```

Các con số trên là mục tiêu tham khảo.

**\*\*Phải benchmark thực tế\*\*, không được giả lập FPS.**

---

## # 10. Performance Metrics

Trong quá trình test hãy đo:

```text

Camera FPS

Stream FPS

AI FPS

YOLO inference time

MediaPipe inference time

Face Recognition inference time

JPEG encoding time

Total processing latency

Dropped frames

CPU usage

Memory usage

```

Có thể log ra terminal.

Không cần tạo UI mới cho performance metrics trong task này.

---

## # 🎯 MỤC TIÊU CHÍNH

Ưu tiên:

```text

VIDEO MƯỢT

>

AI inference tối đa

```

Ví dụ:

```text

Camera ~30 FPS

Stream ~20–30 FPS

YOLO ~10–15 FPS

MediaPipe throttled

Recognition throttled/cache

Phone throttled

```

Không cần tất cả AI chạy 30 FPS.

Quan trọng nhất là camera nhìn **\*\*mượt và realtime\*\***, không còn tình trạng khoảng 1–4 FPS do AI blocking.

---

## # 🛠️ TEST SAU KHI SỬA

Phải kiểm tra:

1. Camera khởi động bình thường.
2. Camera stream bình thường.
3. Một người trước camera.
4. Nhiều người trước camera.
5. Phone detection vẫn hoạt động.
6. Face recognition vẫn hoạt động.
7. MediaPipe vẫn hoạt động.
8. Start Class vẫn hoạt động.
9. End Class vẫn hoạt động.
10. Camera release/restart vẫn hoạt động.
11. Không có memory leak.
12. Không có frame queue tăng vô hạn.

- 13. Socket.IO vẫn hoạt động.
- 14. Các API hiện tại không bị break.

---

### # 📁 FILE CÓ THỂ TẬP TRUNG

Ưu tiên audit và sửa các file liên quan trực tiếp đến camera:

```text

camera_ai.py

classroom_ai.py

tracker.py

face_recognition.py

app.py

```

Có thể inspect toàn repo để hiểu dependency.

Nhưng **\*\*không sửa các module ngoài phạm vi nếu không thật sự cần thiết\*\***.

Nếu một module khác có bug không liên quan trực tiếp đến camera performance:

→ ghi vào báo cáo cuối

→ KHÔNG tự sửa.

---

### # 🚨 QUY TẮC CUỐI CÙNG

Trước khi code:

#### ### Bước 1

Audit pipeline camera hiện tại.

#### ### Bước 2

Xác định chính xác bottleneck.

#### ### Bước 3

Đưa ra benchmark trước khi sửa nếu có thể.

#### ### Bước 4

Implement thay đổi nhỏ và an toàn.

#### ### Bước 5

Benchmark lại sau khi sửa.

#### ### Bước 6

Báo cáo:

```text

File nào đã sửa

Thay đổi gì

FPS trước

FPS sau

AI latency trước

AI latency sau

CPU/RAM trước/sau nếu đo được

Bottleneck còn lại

```

---

### ## 🚫 NHẮC LẠI

Đây là task:

**\*\*CAMERA PERFORMANCE OPTIMIZATION ONLY\*\***

Không redesign hệ thống.

Không sửa identity.

Không sửa Focus Score.

Không sửa database.

Không sửa student management.

Không sửa analytics.

Không sửa seating.

Không redesign UI.

**\*\*Chỉ làm camera mượt hơn, FPS cao hơn và AI pipeline hiệu quả hơn.\*\***

Nếu đã hiểu rõ phạm vi trên, hãy **\*\*tiến hành audit và triển khai kế hoạch tối ưu Camera Performance.\*\***

## Prompt 3

“Bạn đang làm việc trên repo FocusGuard AI.

## MỤC TIÊU LỚN:

Nâng cấp hệ thống AI camera của FocusGuard AI để xử lý ổn định cả:

1. Chế độ 1 học sinh — phân tích thật ổn định, nhận diện chính xác, focus score hợp lý.
2. Chế độ nhiều học sinh — tracking nhiều người cùng lúc, nhận diện từng người, phân tích hành vi riêng cho từng người, không bị nhầm ID.
3. Camera hiển thị mượt khoảng 20–30 FPS nếu phần cứng cho phép, trong khi AI inference được throttle hợp lý.

## QUAN TRỌNG:

- \* Không rewrite toàn bộ project nếu không cần.
- \* Không phá API/backend/frontend hiện tại.
- \* Không xóa các tính năng AI đang có.
- \* Không tạo fake data để giả lập kết quả.
- \* Ưu tiên code ổn định, dễ debug, dễ mở rộng.
- \* Phần cứng mục tiêu hiện tại: Intel Core i5-1135G7 + Intel Xe Graphics + khoảng 15GB RAM, không giả định NVIDIA CUDA.
- \* Trước khi sửa code phải đọc và hiểu architecture hiện tại.

## PHẦN 1 — AUDIT CODE HIỆN TẠI

Đọc và phân tích tối thiểu:

- \* app.py
- \* camera\_ai.py
- \* classroom\_ai.py
- \* tracker.py
- \* face\_recognition.py
- \* session\_manager.py
- \* repository.py
- \* seating\_manager.py
- \* frontend JavaScript liên quan camera/classroom
- \* requirements.txt

Xác định:

1. Camera đang capture frame ở đâu?
2. YOLO được gọi ở đâu và bao nhiêu lần/frame?
3. ByteTrack đang được sử dụng như thế nào?
4. Face recognition được gọi ở đâu?
5. InsightFace đang xử lý full frame hay crop face?
6. MediaPipe được chạy bao nhiêu lần?
7. Focus score được tính theo frame hay theo thời gian?
8. Socket.IO gửi dữ liệu bao nhiêu lần?
9. JPEG encoding nằm ở đâu?
10. Có xử lý AI đồng bộ ngay trong get\_frame() hay không?
11. Track ID có được duy trì ổn định không?
12. student\_id có bị thay đổi khi track bị mất tạm thời không?
13. Một khuôn mặt có thể bị nhận nhầm thành học sinh khác không?
14. Khi có 10–30 học sinh thì bottleneck nằm ở đâu?

Sau audit, đưa ra:

- \* bottleneck chính
- \* lỗi logic nếu có
- \* vấn đề scalability
- \* vấn đề identity tracking
- \* vấn đề performance

SAU ĐÓ MỚI TIẾN HÀNH SỬA.

## PHẦN 2 — KIẾN TRÚC CAMERA MỚI

Thiết kế pipeline theo hướng:

CAMERA

↓

Camera Capture Thread

↓

Latest Frame Buffer

↓

AI Processing Thread

↓

Detection + Tracking

↓

Per-Track Analysis

↓

AI Result Cache

↓

Streaming Thread

↓

Browser

Không được để toàn bộ AI inference chặn camera streaming.

NGUYÊN TẮC:

- \* Camera luôn lấy frame mới nhất.
- \* Không xếp hàng hàng trăm frame cũ.
- \* Nếu AI chưa xử lý xong frame trước thì bỏ frame cũ và lấy frame mới.
- \* Streaming lấy JPEG mới nhất từ cache.
- \* AI và streaming hoạt động độc lập.

### PHẦN 3 — 1 HỌC SINH

Khi chỉ có 1 học sinh trước camera:

Hệ thống phải ưu tiên độ ổn định.

Pipeline:

Camera

- Person Detection
- Track ID
- Face Crop
- Face Recognition
- MediaPipe Face Landmarks
- Head Pose
- EAR / Drowsiness
- Phone Detection
- Temporal Behavior
- Focus Score

YÊU CẦU:

#### ### Identity

Không gọi face recognition full-frame nếu không cần.

Ưu tiên:

person bounding box

- mở rộng nhẹ bounding box
- crop vùng người/mặt
- face detection/recognition trên crop

Nếu track đã được nhận diện:

track\_id → student\_id

thì không cần nhận diện lại liên tục.

Chỉ recognition lại khi:

- \* track mới xuất hiện
- \* track chưa có identity
- \* identity confidence thấp
- \* khuôn mặt thay đổi đáng kể
- \* track bị mất quá lâu rồi xuất hiện lại

Không để mỗi frame tạo một identity mới.

#### ### Face Analysis

MediaPipe xử lý khuôn mặt đã crop.

Không chạy MediaPipe 30 FPS nếu không cần.

Có thể throttle khoảng 5–10 lần/giây.

#### ### Focus Score

KHÔNG tính:

"mỗi frame trừ điểm"

Mà phải tính theo thời gian.

Ví dụ:

focused\_duration

head\_turn\_duration

phone\_duration

drowsiness\_duration

face\_absent\_duration

Sau đó tính score theo thời gian quan sát.

Điều này giúp FPS thay đổi nhưng focus score vẫn ổn định.

#### =====

#### PHẦN 4 — MULTI-STUDENT TRACKING

=====

Đây là phần quan trọng nhất.

Mục tiêu:

Một camera có thể nhìn thấy nhiều học sinh cùng lúc.

Ví dụ:

Student A

Student B

Student C

Student D

...

Student 30

Mỗi người phải có:

track\_id

student\_id

name

bbox

identity\_confidence

focus\_score

current\_state

head\_pose

phone\_detected

drowsiness\_signal

face\_visible

last\_seen

observation\_history

Pipeline:

YOLO Person Detection

↓

ByteTrack

↓

track\_id

↓

Person BBox

↓

Face Crop

↓

Identity Matching

↓

student\_id

↓

Per-Student AI State

↓

Focus Score

QUAN TRỌNG:

Không được dùng một biến global kiểu:

current\_student

để lưu trạng thái AI.

Mỗi track phải có state riêng.

Ví dụ:

```
tracks = {
 track_1: {
 student_id: "HS001",
 focus_score: 92,
 state: "Focused"
 },
 ...

 track_2: {
 student_id: "HS002",
 focus_score: 76,
 state: "Distracted"
 }
 ...
}
```

Mỗi học sinh phải được xử lý độc lập.

## PHẦN 5 — TRACK ID + IDENTITY

Phân biệt rõ:

`track_id`  $\neq$  `student_id`

`track_id`:

ID tạm thời của ByteTrack.

`student_id`:

ID cố định của học sinh trong hệ thống.

Ví dụ:

`track_id` = 7

`student_id` = HS023

`name` = Nguyễn A

Nếu ByteTrack vẫn giữ track 7:

không được recognition lại liên tục.

Nếu track mất vài frame:

giữ identity trong một khoảng thời gian ngắn.

Nếu track mất quá lâu:

xóa track state.

Khi xuất hiện lại:

thực hiện recognition lại.

## PHẦN 6 — CHỐNG NHẦM ID

Đây là yêu cầu bắt buộc.

Không được:

Frame 1:

track 5  $\rightarrow$  Nguyễn A

Frame 2:

track 5  $\rightarrow$  Nguyễn B

Frame 3:

track 5  $\rightarrow$  Nguyễn A

vì recognition dao động.

Phải có identity stabilization.

Ví dụ:

```
identity_candidates = [
 Nguyễn A,
 Nguyễn A,
 Nguyễn A,
 Nguyễn B
]
```

$\rightarrow$  giữ Nguyễn A.

Có thể sử dụng:

- \* confidence threshold
- \* cosine similarity threshold
- \* temporal voting
- \* identity cache
- \* minimum recognition confidence
- \* track persistence

Không đổi identity chỉ vì một frame recognition khác.

## PHẦN 7 — FACE RECOGNITION SCALABILITY

Không được chạy InsightFace full-frame cho toàn bộ camera ở mọi frame.

Thay vào đó:

YOLO/ByteTrack tìm person

→ lấy bbox

→ crop vùng đầu/mặt

→ recognition trên crop

Recognition phải được throttle.

Ví dụ:

1 học sinh:

3–5 recognition/s

10 học sinh:

không nhất thiết 30–50 recognition/s.

Có thể round-robin:

Frame N:

track 1, 2

Frame N+1:

track 3, 4

Frame N+2:

track 5, 6

...

Nhưng phải đảm bảo track mới/chưa xác định được ưu tiên.

## PHẦN 8 — MEDIAPIPE MULTI-FACE

Không chạy MediaPipe toàn bộ camera ở 30 FPS cho 30 học sinh.

Thay vào đó:

Mỗi track có thời gian xử lý riêng:

track\_1 → last\_mediapipe\_time

track\_2 → last\_mediapipe\_time

track\_3 → last\_mediapipe\_time

Nếu:

$\text{current\_time} - \text{last\_mediapipe\_time} > \text{interval}$

thì xử lý track đó.

Khoảng mục tiêu:

5–10 Hz/person

nhưng phải adaptive theo performance.

Nếu 30 học sinh:

ưu tiên:

\* người đang rõ mặt

\* người đang có track ổn định

\* người mới xuất hiện

\* người có observation đang cần xác nhận

Không cần xử lý mọi người ở mọi frame.

## PHẦN 9 — PHONE DETECTION

Phone detection không cần chạy mỗi frame.

Có thể throttle khoảng:

3-8 FPS

Nhưng kết quả phải được giữ trong cache.

Ví dụ:

track\_7:

phone\_detected = True

thì giữ trạng thái trong một khoảng thời gian hợp lý.

Không để:

Frame 1 → phone

Frame 2 → no phone

Frame 3 → phone

Frame 4 → no phone

làm UI nhấp nháy liên tục.

Dùng temporal smoothing / short persistence.

## PHẦN 10 — TEMPORAL BEHAVIOR

Không kết luận hành vi chỉ dựa trên một frame.

Ví dụ:

Yaw > threshold trong 1 frame

KHÔNG ngay lập tức:

"Distracted"

Thay vào đó:

head\_turn\_start

→ duration

→ confidence

→ observation

Ví dụ:

Head turned for 0.8s

Head turned for 2.1s

Head turned for 5.2s

Sau đó mới tạo observation phù hợp.

Tương tự:

\* drowsiness

\* phone

\* face absent

\* looking away

Mỗi track có temporal state riêng.

## PHẦN 11 — FOCUS SCORE

Thiết kế lại focus score để:

\* ổn định

\* giải thích được

\* không phụ thuộc trực tiếp vào FPS

\* không dao động mạnh

\* tính riêng cho từng học sinh

Ví dụ dữ liệu:

focus\_duration

looking\_away\_duration

phone\_duration

drowsiness\_duration

face\_absent\_duration

Score phải dựa trên duration / weighted observations.

Không được dùng:

score -= x

ở mỗi frame mà không xét elapsed time.

Nên có:

last\_update\_time

delta\_time

để score thay đổi dựa trên thời gian thực.

Ví dụ:

nếu học sinh focused:

$\text{recovery\_rate} \times \text{delta\_time}$

nếu distracted:

$\text{penalty\_rate} \times \text{delta\_time}$

## PHẦN 12 — AI RESULT CACHE

Tạo một state/cache trung tâm.

Ví dụ:

```
student_states = {
 track_id: {
 student_id,
 name,
 bbox,
 focus_score,
 state,
 head_pose,
 phone_detected,
 drowsiness,
 face_visible,
 identity_confidence,
 last_seen,
 last_recognition,
 last_mediapipe,
 last_phone_detection
 }
}
```

Cache phải thread-safe.

Streaming chỉ đọc cache.

AI processor cập nhật cache.

Frontend không được phụ thuộc trực tiếp vào model inference.

## PHẦN 13 — CAMERA STREAM

Mục tiêu:

Camera display khoảng 20–30 FPS nếu hardware cho phép.

Không yêu cầu AI inference 30 FPS.

Ví dụ:

Camera:

30 FPS

YOLO:

10–15 FPS

Phone:

3–8 FPS

Face Recognition:

3–5 FPS/person hoặc khi cần

MediaPipe:

5–10 FPS/person

Socket.IO:

5–10 updates/sec hoặc event-based

Stream:

20–30 FPS

Nếu hardware không đạt được thì tự động giảm resolution / inference frequency thay vì làm app treo.

## PHẦN 14 — ADAPTIVE PERFORMANCE

Hệ thống phải tự điều chỉnh theo số lượng học sinh.

Ví dụ:  
1–3 người:  
AI đầy đủ hơn.  
4–10 người:  
throttle nhẹ.  
10–20 người:  
round-robin + caching.  
20–30 người:  
ưu tiên tracking + identity + essential behavior.  
Không được để số người tăng làm FPS giảm thẳng xuống 1 FPS nếu có thể tránh.  
Tạo performance metrics:  
camera\_fps  
stream\_fps  
ai\_fps  
yolo\_fps  
phone\_fps  
recognition\_fps  
mediapipe\_fps  
average\_inference\_ms  
active\_tracks  
recognized\_students  
Có thể hiển thị các metrics này trong debug mode.

## PHẦN 15 — DRAWING

Trên camera stream:  
Mỗi người:  
Bounding Box  
Name  
Focus Score  
State  
Ví dụ:

Nguyễn An	
Focus 92	
Focused	

Không vẽ classroom grid lên camera.  
Không làm UI camera rối.  
Nếu chưa nhận diện:  
Unknown  
Focus ---  
Không tự gán tên dựa trên guess.

## PHẦN 16 — CLASSROOM DATA

Teacher dashboard phải nhận được danh sách đầy đủ:  
student\_id  
name  
status  
focus\_score  
current\_state  
track\_id  
last\_seen  
Ví dụ:

HS001 | Nguyễn An | Online | 92 | Focused  
HS002 | Trần Bình | Online | 76 | Looking Away  
HS003 | Lê Minh | Away | -- | Face Not Visible  
Không chỉ hiển thị những người đang nằm trong frame hiện tại.

Student list phải lấy từ class/session data.  
Camera tracking chỉ cập nhật trạng thái của student.

## PHẦN 17 — SESSION

Khi Start Class:

- \* reset focus score về giá trị khởi đầu
- \* reset temporal behavior state
- \* reset current activity log
- \* reset active track state
- \* reset temporary recognition state nếu cần

Nhưng:

KHÔNG xóa session cũ.

Session cũ phải được archive/save để Analytics và Reports có thể xem lại.

Khi End Class:

- \* đóng session
- \* lưu summary
- \* lưu per-student statistics
- \* lưu activity/observation history
- \* giải phóng camera/AI resources đúng cách

## PHẦN 18 — SOCKET.IO

Không gửi toàn bộ frame/image qua Socket.IO nếu HTTP/MJPEG/WebSocket stream đã xử lý video.

Socket.IO chỉ gửi metadata:

student\_update

class\_started

class\_ended

observation\_created

focus\_update

attendance\_update

Throttle update nếu cần.

Không spam socket 30 lần/giây cho từng học sinh nếu không cần.

## PHẦN 19 — ERROR HANDLING

Nếu:

camera disconnect

model load fail

face recognition fail

MediaPipe fail

một track lỗi

thì không được crash toàn bộ classroom.

Ví dụ:

MediaPipe fail cho track 7

→ chỉ track 7 thiếu face analysis

→ các track khác vẫn chạy.

Recognition fail

→ giữ identity cũ nếu còn hợp lệ

→ không crash.

Camera reconnect:

→ thử reconnect an toàn.

## PHẦN 20 — TEST PLAN

Sau khi code xong phải test:

TEST 1:

1 học sinh, nhìn thẳng.

TEST 2:

1 học sinh quay đầu.  
TEST 3:  
1 học sinh rời camera.  
TEST 4:  
1 học sinh quay lại.  
TEST 5:  
2 học sinh.  
TEST 6:  
5 học sinh.  
TEST 7:  
10+ học sinh nếu hardware cho phép.  
TEST 8:  
Hai người đi ngang nhau.  
TEST 9:  
Một người che mặt tạm thời.  
TEST 10:  
Một người biến mất vài giây rồi xuất hiện lại.  
TEST 11:  
Hai người có ngoại hình tương tự.  
TEST 12:  
Phone detection.  
TEST 13:  
Drowsiness/head pose.  
TEST 14:  
Start Class → End Class → Start Class lại.  
TEST 15:  
Kiểm tra FPS khi tăng số lượng học sinh.  
TEST 16:  
Camera disconnect/reconnect.

=====

## PHẦN 21 — ACCEPTANCE CRITERIA

=====

Sau khi hoàn thành:

### ### 1 STUDENT

- \* nhận diện ổn định
- \* track ổn định
- \* không đổi tên liên tục
- \* focus score vượt
- \* behavior có temporal smoothing
- \* camera không bị AI inference khóa cứng

### ### MULTI-STUDENT

- \* nhiều người có track\_id riêng
- \* mỗi track có state riêng
- \* identity mapping ổn định
- \* không dùng chung focus score
- \* không nhầm state giữa người này và người kia
- \* mất/hiện lại người được xử lý đúng
- \* số lượng người tăng không làm hệ thống crash

### ### PERFORMANCE

Không chỉ báo "FPS = 30" bằng cách fake counter.

FPS phải được đo thực tế.

Nếu hardware không đạt 30 FPS:

- \* báo FPS thực tế
- \* xác định bottleneck
- \* tối ưu tiếp
- \* giảm inference frequency/resolution nếu cần.

### ### QUALITY

Không dùng fake recognition.

Không random score.

Không random student.  
Không hardcode kết quả AI.

=====

## PHẦN 22 — SAU KHI CODE XONG

=====

Không chỉ nói "done".

Hãy trả về report:

1. Files đã thay đổi.
2. Architecture trước.
3. Architecture sau.
4. Bottleneck chính đã xử lý.
5. Cách multi-student tracking hoạt động.
6. Cách track\_id → student\_id hoạt động.
7. Cách identity stabilization hoạt động.
8. Cách MediaPipe được throttle.
9. Cách phone detection được throttle.
10. Cách focus score được tính theo thời gian.
11. FPS trước/sau nếu đo được.
12. Số người đã test.
13. Những giới hạn còn lại.
14. Những phần KHÔNG được thay đổi.

Đặc biệt:

KHÔNG được tự ý thay đổi UI lớn trong task này.

Task này tập trung vào:

CAMERA ENGINE

+

1-STUDENT ANALYSIS

+

MULTI-STUDENT TRACKING

+

IDENTITY STABILITY

+

TEMPORAL BEHAVIOR

+

FOCUS SCORE

+

PERFORMANCE

Giữ nguyên backend/API hiện tại nếu không cần thay đổi.

Nếu phát hiện architecture hiện tại đã có một phần tốt, hãy reuse thay vì viết lại.

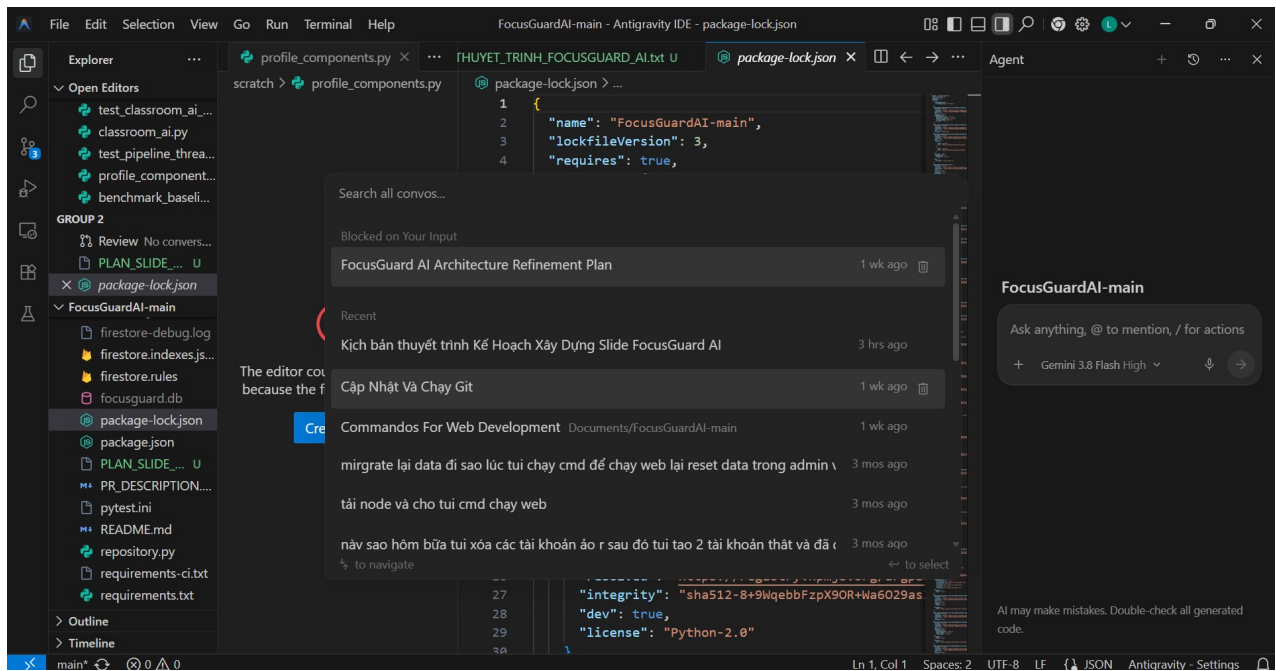
Trước khi sửa file, hãy audit và lập implementation plan ngắn gọn.

Sau đó mới thực hiện code.

nhớ làm 1 plan để tui coi r mới proceed”

**Ghi chú của thí sinh:** Vì đa số em lấy prompt từ chatgpt nên prompt khá dài nên em chỉ đưa lên 3 prompt mong ban tổ chức thông cảm ạ

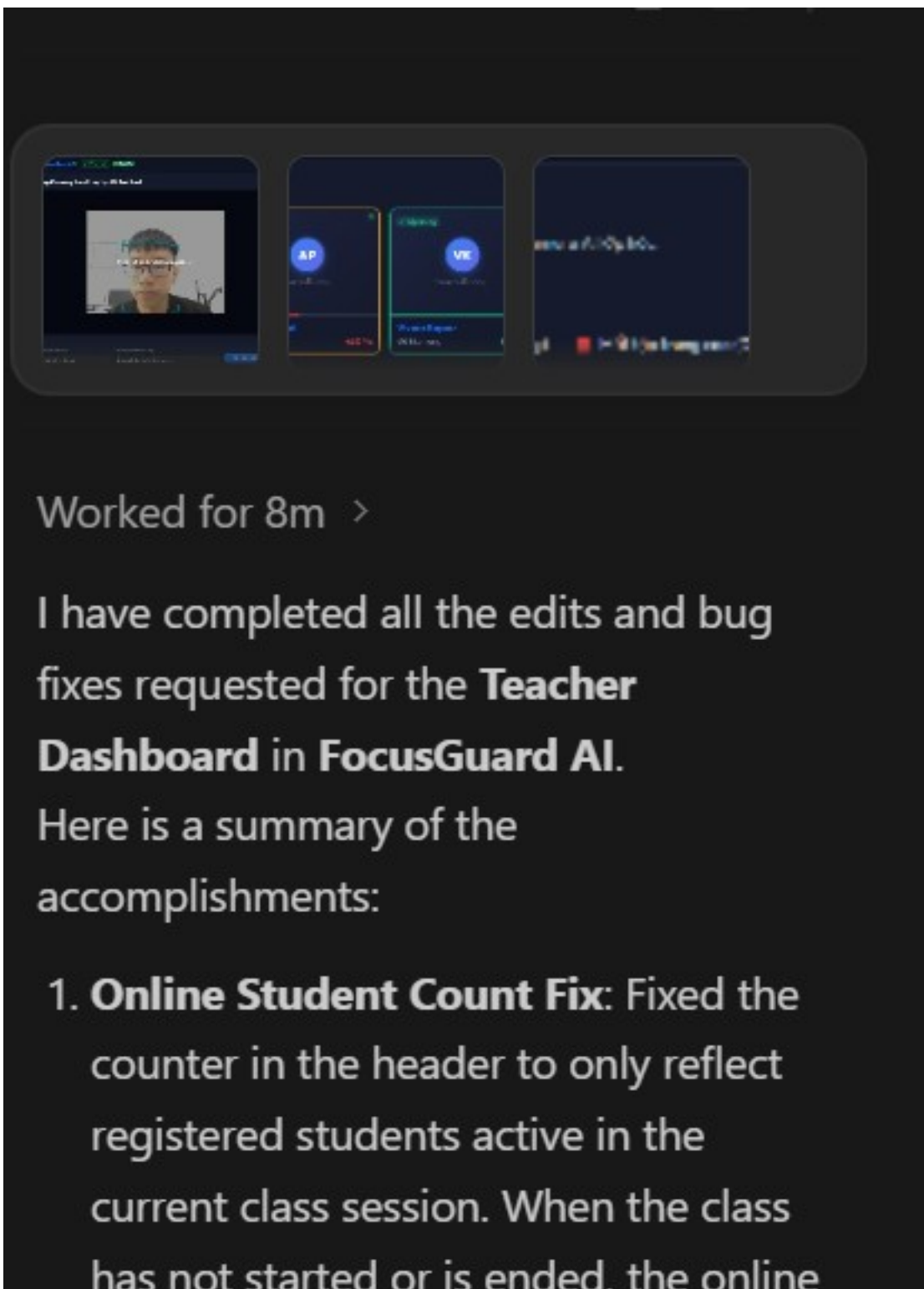
## 2. Hình ảnh minh chứng quá trình sử dụng AI và phát triển sản phẩm



Hình 1. Lịch sử các cuộc hội thoại phát triển FocusGuard trong môi trường lập trình, thể hiện các chủ đề kiến trúc, cập nhật Git và xử lý dữ liệu.



Hình 2. Minh chứng trợ lý AI lập kế hoạch triển khai chức năng Classroom AI Monitoring trước khi chỉnh sửa mã nguồn.



Hình 3. Minh chứng quá trình phát triển/đổi chiều giao diện FocusGuard trong phiên làm việc với trợ lý AI.

mirgrate lại data đi sao lúc tui chạy cmd để chạy web lại reset data trong admin v lúc đó tui xóa tài khoản ảo r tạo tài khoản mới xong tắt cmd tui mở lại r reset

Worked for 1m >

Nguyên nhân dữ liệu của bạn bị mất và reset về ban đầu là do cách tắt cửa sổ dòng lệnh (CMD).

### Nguyên nhân chi tiết

1. **Chạy trên RAM:** Firebase Emulator chạy cơ sở dữ liệu **hoàn toàn trên RAM** để phát triển cục bộ.
2. **Không chạy kịp tiến trình lưu (Export):** Khi bạn khởi chạy bằng `run_app.bat`, hệ thống sử dụng lệnh `--export-on-exit=./data/firebase` để tự động lưu dữ liệu từ RAM xuống ổ đĩa khi tắt. Tuy nhiên, lệnh này **chỉ**



Review Changes

Hình 4. Minh chứng quá trình rà soát và verification sau khi triển khai, dùng để đối chiếu với sản phẩm thật.

### 3. Minh chứng kỹ thuật có thể đối chiếu trong repository

Minh chứng	Nội dung có thể đối chiếu
README.md	Kiến trúc, trạng thái test, hạn chế và cách chạy
SUMMARY.md	Competition hardening và trạng thái chức năng
docs/COMPETITION_ARCHITECTURE.md	Identity, behavior, FocusEngine, runtime, realtime, authz
docs/REAL_CAMERA_RESULTS.md	Môi trường camera thật, kịch bản PASS và lỗi đã sửa
docs/PRIVACY_AND_AI_SAFETY.md	Dữ liệu sinh trắc, không ghi video, phạm vi sử dụng
.github/workflows/ci.yml	CI compile/import/pytest/JS/security/evaluation
Git history	Commit/PR thể hiện quá trình sửa lỗi và kiểm thử

## 4. Công cụ, thư viện và trợ lý AI đã sử dụng

Công cụ / thư viện	Vai trò được kê khai
YOLOv8 / Ultralytics	Mô hình/thư viện có sẵn; đội tích hợp và kiểm thử
ByteTrack	Tracker có sẵn; track_id chỉ dùng tạm
MediaPipe Face Landmarker	Mô hình có sẵn cho landmark
InsightFace	Mô hình có sẵn cho embedding khuôn mặt
Flask / Socket.IO	Framework backend/realtime
Firebase Admin / Firestore	Backend tùy chọn; client bị deny-all
ChatGPT	Phân tích yêu cầu, hỗ trợ xây prompt, rà soát hồ sơ và giải thích
Claude Code	Đọc/sửa mã nguồn theo prompt, chạy test, audit và verification

**Thư mục minh chứng gốc trên Google Drive:**

<https://drive.google.com/drive/folders/1K6HAI9LngnBFuSgdzzg4nP0t6cMwB042>